

The Autoregressive Bottleneck, Worked Out by Hand

WHY ONE GENERATED TOKEN COSTS SO MUCH — A COMPLETE NUMERICAL WALK-THROUGH

What this document does. It takes one concrete serving target — the *Nexus* fraud-detection assistant, a 70-billion-parameter decoder running on a single NVIDIA H100 — and computes, from first principles, the *floor* on how fast it can emit a token. Nothing is asserted without arithmetic. Every number below is produced by a small reference script (Appendix A, `m01_t01_engine.py`) so the figures are exact and reproducible. By the end you will be able to derive, on paper, why a single H100 cannot push this model below ~ 42 ms per token, and why that number — not FLOPs — is the thing every later module attacks.

The one idea. An LLM in *decode* is not compute-limited; it is *memory-bandwidth*-limited. The bottleneck is not "how much math," it is "how many bytes of weights must cross the memory bus to produce the next token." We will make that claim quantitative.

CONCEPTS COVERED, IN ORDER

the two phases of inference (prefill vs. decode); the weight-memory footprint; the decode latency floor $t_{\min} = W_{\text{bytes}}/B_{\text{HBM}}$; arithmetic intensity; the roofline ridge point; why decode runs at $\sim 0.3\%$ of peak FLOPs; a layer-by-layer sanity check; and the three escape routes.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The system under study	2
2	Step 1 — The two phases: prefill vs. decode	2
3	Step 2 — The weight-memory footprint	3
4	Step 3 — The decode latency floor	3
5	Step 4 — <i>Why</i> decode is memory-bound: arithmetic intensity	4
6	Step 5 — A layer-by-layer sanity check	5
7	Step 6 — The three escape routes	5
8	An honest word about this floor	5
9	The argument, at a glance	6
9.1	Equation cheat-sheet	6

Appendix A — Reference implementation

6

1 The system under study

Real production models span $d_{\text{model}} = 4096$ to 16384 and dozens of layers. We fix one concrete instance so every quantity is a number you can check.

Quantity	Symbol	Value here
Parameters	P	70×10^9
Precision	—	FP16 (2 bytes/param)
Hidden width	d_{model}	8192
Decoder layers	N_{layers}	80
HBM bandwidth (H100 SXM, HBM3)	B_{HBM}	$3.35 \text{ TB/s} = 3.35 \times 10^{12} \text{ B/s}$
FP16 tensor-core peak (H100)	F_{peak}	$990 \text{ TFLOP/s} = 9.9 \times 10^{14} \text{ FLOP/s}$
Prompt length	T_{prompt}	6200 tokens
Output length	N_{out}	180 tokens

We use decimal units throughout: $1 \text{ GB} = 10^9 \text{ B}$, $1 \text{ TB/s} = 10^{12} \text{ B/s}$ — the same convention vendors use when they say "140 GB model" or "3.35 TB/s."

Intuition

A decoder generates text *autoregressively*: to produce token n it must first have produced token $n - 1$. There is no way to compute token 5 and token 6 at the same time on a single stream — token 6's input *is* token 5. So a 180-token answer is 180 strictly sequential forward passes. The cost of the whole answer is (number of tokens) $\times$ (cost of one forward pass). This document is about that second factor.

2 Step 1 — The two phases: prefill vs. decode

Every request runs the same weights in two very different regimes.

PREFILL • "fill the context"

Process all 6200 prompt tokens *at once*.

Per layer: $X_{[6200 \times 8192]} \cdot W_{[8192 \times 8192]}$
 $\Rightarrow$ big matrix $\times$ matrix.

Each weight read once, reused by 6200 rows.

Regime: COMPUTE-bound.

Tensor cores saturated; limit is F_{peak} .

DECODE • "one token at a time"

Produce each of 180 tokens, one by one.

Per layer, per token: $x_{[1 \times 8192]} \cdot W_{[8192 \times 8192]}$
 $\Rightarrow$ tiny vector $\times$ huge matrix.

Each weight read once, reused by 1 row.

Regime: MEMORY-bound.

Cores idle, waiting on HBM; limit is B_{HBM} .

The two boxes run *identical weights*. The only thing that changed is the number of rows in the activation: 6200 in prefill, 1 in decode. That single difference flips the bottleneck from compute to memory — and Steps 3–4 show exactly why.

The single most important insight in LLM serving

Prefill is compute-bound. Decode is memory-bandwidth-bound. They are different workloads on the same silicon. Every optimization in this course either (a) makes prefill do less math, (b) makes decode read fewer bytes, or (c) hides one phase behind the other.

3 Step 2 — The weight-memory footprint

To run a forward pass you must read the weights. How many bytes is that?

Footprint, step by step

$P = 70 \times 10^9$ parameters, each stored in FP16 (2 bytes):

$$W_{\text{bytes}} = P \times (\text{bytes/param}) = 70 \times 10^9 \times 2 = 1.4 \times 10^{11} \text{ B} = 140 \text{ GB}.$$

Mini-example — the operation in isolation

”Bytes from a count and a width” is just multiplication. A 7 B model in INT8 (1 byte) is 7 GB; the *same* model in FP16 is 14 GB; in FP32, 28 GB. Precision is a literal multiplier on every byte you must move. This is the entire reason quantization (Module 02) exists.

4 Step 3 — The decode latency floor

Here is the load-bearing argument of the whole module.

To produce *one* decode token, the model must — at minimum — read every weight exactly once during the forward pass. (You cannot multiply by a weight you have not loaded; and with batch = 1 there is no other request to share that load with.) So a hard lower bound on the time per token is ”how long does it take to stream W_{bytes} through the memory bus?”

Decode time lower bound — memory-bound regime

$$t_{\min} = \frac{W_{\text{bytes}}}{B_{\text{HBM}}}.$$

Plug in the numbers

$$t_{\min} = \frac{1.4 \times 10^{11} \text{ B}}{3.35 \times 10^{12} \text{ B/s}} = 4.179 \times 10^{-2} \text{ s} = 41.8 \text{ ms per token}.$$

Full answer: $N_{\text{out}} \cdot t_{\min} = 180 \times 41.8 \text{ ms} = 7.52 \text{ s}$ of pure decode. ✓

This is a *floor*, assuming an impossible 100% bandwidth utilization. Real serving is slower, never faster, on this hardware at batch 1.

Intuition

Think of the 140 GB of weights as a book you must read *cover to cover* to write each new word, and HBM bandwidth as how fast you can turn pages. The H100 turns 3.35 TB worth of pages per second, so one full read of the book takes 42 ms. It does not matter

that the actual "thinking" (the FLOPs) for one token is trivial — you still had to read the whole book to do it. The page-turning, not the thinking, sets the pace.

5 Step 4 — Why decode is memory-bound: arithmetic intensity

The claim "decode is memory-bound" is not a vibe; it is a ratio. **Arithmetic intensity** (AI) is FLOPs of useful work divided by bytes of memory traffic:

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}} \quad [\text{FLOP/B}].$$

Consider one weight matrix $W \in \mathbb{R}^{d \times d}$ with $d = d_{\text{model}} = 8192$. Reading it once costs $d^2 \cdot 2 = 1.342 \times 10^8 \text{ B}$ (=134 MB). A matmul of an $[n \times d]$ activation against it costs $2nd^2$ FLOPs.

The same matrix, two regimes

	rows n	FLOPs = $2nd^2$	AI = FLOPs/134 MB
decode	1	1.342×10^8	1.0 FLOP/B
prefill	6200	8.321×10^{11}	6200 FLOP/B

The bytes are *identical* in both rows — you read W once either way. Only the work done with those bytes differs. Decode does 1 FLOP per byte read; prefill does 6200.

Whether 1 FLOP/B is "a lot" or "a little" is decided by the hardware's **ridge point** — the AI at which a chip's compute and memory roofs cross:

Ridge point — where memory-bound flips to compute-bound

$$\text{ridge} = \frac{F_{\text{peak}}}{B_{\text{HBM}}} = \frac{9.9 \times 10^{14}}{3.35 \times 10^{12}} = 295.5 \text{ FLOP/B}.$$

Now compare. Achievable throughput on a roofline is $\min(\text{AI} \times B_{\text{HBM}}, F_{\text{peak}})$:

Regime	AI (FLOP/B)	vs. ridge 295.5	Achievable
Decode	1.0	$\ll \text{ridge} \Rightarrow$ memory-bound	$3.35 \times 10^{12} \text{ FLOP/s} = \mathbf{0.34\%}$ of peak
Prefill	6200	$\gg \text{ridge} \Rightarrow$ compute-bound	$9.9 \times 10^{14} \text{ FLOP/s} = \mathbf{100\%}$ of peak

The number that explains the whole course

In decode, the H100's tensor cores run at **0.34% of their rated FLOP/s**. They are not slow — they are *starved*, sitting idle 99.66% of the time waiting for weights to arrive over HBM. Buying more FLOPs (a faster compute chip) does nothing. The only levers that move decode latency are: read *fewer* bytes, or read the *same* bytes for *more* useful work.

Mini-example — why scaling by $\sqrt{d}$ is not the point here

A common reflex is "make the math cheaper." But decode's math is already 0.34% utilized; halving the FLOPs changes a number that is not the bottleneck. The bottleneck is the 134 MB per matrix you must fetch. Halve *that* (e.g. INT8) and you halve the time. Optimize the binding constraint, not the slack one.

6 Step 5 — A layer-by-layer sanity check

The floor in Step 3 treated the model as one 140 GB blob. It should also be the sum of its layers. With weights spread evenly over 80 layers:

$$\begin{aligned}\text{per-layer bytes} &= \frac{140 \text{ GB}}{80} = 1.75 \text{ GB}, \\ \text{per-layer time} &= \frac{1.75 \times 10^9}{3.35 \times 10^{12}} = 0.522 \text{ ms}, \\ \text{total} &= 80 \times 0.522 \text{ ms} = 41.8 \text{ ms.} \quad \checkmark\end{aligned}$$

The two derivations agree to the rounding. Good — the floor is not an accounting trick of lumping everything together; it survives decomposition.

7 Step 6 — The three escape routes

If $t_{\min} = W_{\text{bytes}}/B_{\text{HBM}}$, then every way to go faster must change a term in that fraction. There are exactly three, and the rest of the course is a tour of them.

Route	What it changes	Effect on $t_{\min}$
Read fewer weights	Shrink W_{bytes} : quantization (INT8/INT4), pruning, distillation, MoE (skip experts).	INT4 $\Rightarrow$ 35 GB $\Rightarrow$ 10.4 ms
Reuse each read	Amortize one weight read over a <i>batch</i> of requests; the 42 ms becomes a <i>per-step</i> cost shared by all.	$B=32 \Rightarrow$ 1.3 ms/token-equiv
Hide the read	Overlap memory traffic with compute: speculative decoding, chunked prefill, pipelining.	multiple tokens per read

Watch out

The "batch" route changes *throughput per GPU*, not single-stream *latency*: each individual user still waits ~ 42 ms per step, but the GPU now serves 32 of them in that time. Latency and throughput are different objectives, and confusing them is the most common mistake in capacity planning (Module 05, Module 08). If someone claims to "make inference faster" without touching W_{bytes} , precision, batch size, or hardware, they are optimizing a metric that does not move single-stream decode latency.

8 An honest word about this floor

Watch out

$t_{\min} = 42$ ms assumes 100% HBM utilization, which never happens: real kernels hit 60–85% of peak bandwidth, so observed per-token time is more like 50–70 ms at batch 1. The floor is still the right mental model — it tells you the *best case* and which resource you are spending. You cannot beat it by writing cleverer kernels; you can only approach it. Beating it requires changing the physics: fewer bytes, shared bytes, or hidden bytes. That is the whole game.

9 The argument, at a glance

#	Step	Result
1	Two phases	prefill = compute-bound; decode = memory-bound
2	Footprint	$W_{\text{bytes}} = P \cdot 2 = 140 \text{ GB}$
3	Floor	$t_{\text{min}} = W_{\text{bytes}} / B_{\text{HBM}} = 41.8 \text{ ms/token}$
4	Why	decode AI = 1 $\ll$ ridge 295.5 $\Rightarrow$ 0.34% of peak FLOPs
5	Check	$80 \times 0.522 \text{ ms} = 41.8 \text{ ms}$ ✓
6	Escapes	fewer bytes / shared bytes / hidden bytes

9.1 Equation cheat-sheet

Weight footprint	$W_{\text{bytes}} = P \times (\text{bytes/param})$
Decode floor	$t_{\text{min}} = W_{\text{bytes}} / B_{\text{HBM}}$
Arithmetic intensity	AI = FLOPs/bytes moved
Ridge point	ridge = $F_{\text{peak}} / B_{\text{HBM}}$
Achievable FLOP/s	$\min(\text{AI} \times B_{\text{HBM}}, F_{\text{peak}})$
Bound test	AI < ridge $\Rightarrow$ memory-bound

Appendix A — Reference implementation

Every number above is produced by `m01_t01_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change P , B_{HBM} , or F_{peak} to explore other models and accelerators.

```

1  """
2  Reference implementation for: Module 01 - Topic 1
3  "The Autoregressive Bottleneck - Why one token costs so much."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  Convention: 1 GB = 1e9 bytes, 1 TB = 1e12 bytes, 1 TB/s = 1e12 B/s (decimal,
9  the convention vendors use for HBM bandwidth and model "size in GB").
10 """
11
12 # -----
13 # 0. The system under study: one 70B decoder on one H100 SXM.
14 # -----
15 P          = 70e9          # parameters (Llama-3-70B class)
16 bytes_fp16 = 2             # bytes per parameter at FP16
17 d_model    = 8192          # hidden width
18 n_layers   = 80
19 B_hbm      = 3.35e12        # H100 SXM HBM3 peak bandwidth, bytes/s
20 F_peak     = 990e12         # H100 FP16 tensor-core peak, FLOP/s
21 T_prompt   = 6200          # Nexus prompt length (tokens)
22 N_out      = 180           # Nexus output length (tokens)
23
24 # -----
25 # 1. Weight memory footprint.
26 # -----
27 W_bytes = P * bytes_fp16
28 print("== 1. Weight footprint ==")
29 print(f"W_bytes = {P:.0e} x {bytes_fp16} B = {W_bytes:.3e} B = {W_bytes/1e9:.0f} GB")
30
31 # -----

```

```

32 # 2. Decode lower bound: must stream every weight once per token.
33 # -----
34 t_min = W_bytes / B_hbm # seconds / token
35 t_min_ms = t_min * 1e3
36 print("\n== 2. Decode lower bound (memory-bound floor) ==")
37 print(f"t_min = W_bytes / B_hbm = {W_bytes:.3e} / {B_hbm:.3e} = {t_min:.5f} s = {t_min_ms:.1f} ms/token")
38 print(f"180 tokens -> {N_out * t_min:.2f} s of pure decode (floor, single H100, batch=1)")
39
40 # -----
41 # 3. Arithmetic intensity of ONE projection, decode vs prefill.
42 # A single weight matrix W in  $\mathbb{R}^{d \times d}$ ; read once =  $d \times d \times 2$  bytes.
43 # matmul of an  $[n \times d]$  activation by  $[d \times d]$  costs  $2 \times n \times d \times d$  FLOPs.
44 # -----
45 def matmul_flops(n, d): return 2 * n * d * d
46 def weight_bytes(d): return d * d * bytes_fp16
47
48 w_b = weight_bytes(d_model)
49 print("\n== 3. Arithmetic intensity of one  $d \times d$  projection ==")
50 print(f"weights read once =  $d^2 \times 2B = \{w_b:.3e\} B = \{w_b/1e6:.0f\} MB$ ")
51
52 for name, n in [("decode (1 token)", 1), ("prefill (6200 tokens)", T_prompt)]:
53     flops = matmul_flops(n, d_model)
54     ai = flops / w_b
55     print(f" {name:24s}: FLOPs={flops:.3e} AI={ai:7.2f} FLOP/B")
56
57 # -----
58 # 4. The ridge point: where memory-bound flips to compute-bound.
59 # -----
60 ridge = F_peak / B_hbm
61 print("\n== 4. Ridge point of the H100 ==")
62 print(f"ridge = F_peak / B_hbm = {F_peak:.2e} / {B_hbm:.2e} = {ridge:.1f} FLOP/B")
63 ai_decode = matmul_flops(1, d_model) / w_b
64 ai_prefill = matmul_flops(T_prompt, d_model) / w_b
65 print(f"decode AI={ai_decode:.2f} vs ridge {ridge:.0f} -> {'COMPUTE' if ai_decode>ridge else 'MEMORY'}-bound")
66 print(f"prefill AI={ai_prefill:.0f} vs ridge {ridge:.0f} -> {'COMPUTE' if ai_prefill>ridge else 'MEMORY'}-bound")
67
68 # -----
69 # 5. Achievable throughput from the roofline:  $\min(AI \times B, F_{\text{peak}})$ .
70 # -----
71 def achievable(ai): return min(ai * B_hbm, F_peak)
72 print("\n== 5. Achievable FLOP/s (roofline) ==")
73 for name, ai in [("decode", ai_decode), ("prefill", ai_prefill)]:
74     ach = achievable(ai)
75     print(f" {name:8s}:  $\min(\{ai:.2f\} \times \{B_hbm:.2e\}, \{F_{\text{peak}}:.2e\}) = \{ach:.3e\} \text{ FLOP/s}$  "
76           f" $\{100 \times ach / F_{\text{peak}}:.2f\} \% \text{ of peak}$ ")
77
78 # -----
79 # 6. Per-token decode time built up layer by layer (sanity vs the floor).
80 # Every layer must read its slice of the 140 GB exactly once.
81 # -----
82 per_layer_bytes = W_bytes / n_layers
83 per_layer_ms = per_layer_bytes / B_hbm * 1e3
84 print("\n== 6. Layer-by-layer decode time ==")
85 print(f"per-layer weights = {per_layer_bytes/1e9:.3f} GB -> {per_layer_ms:.3f} ms/layer/token")
86 print(f"x {n_layers} layers = {per_layer_ms*n_layers:.1f} ms/token (matches t_min={t_min_ms:.1f})")
87
88 # -----
89 # 7. The three escape routes, as multipliers on t_min.
90 # -----
91 print("\n== 7. Escape routes (illustrative multipliers) ==")
92 print(f" baseline FP16 : {t_min_ms:5.1f} ms/token")
93 print(f" read fewer weights (INT4): {W_bytes/4/B_hbm*1e3:5.1f} ms/token (4x less to read)")
94 print(f" batch B=32 (amortize) : {t_min_ms:5.1f} ms/STEP shared by 32 reqs -> {t_min_ms/32:4.2f} ms/token-")

```

— END OF DOCUMENT —